

МЕKF для оценки ориентации по кватернионам изображения

Подробный конспект по математической модели, алгоритму и программе из запроса. Рассматривается вариант без гироскопа: фильтр получает кватернион ориентации от алгоритма компьютерного зрения и дополнительно оценивает угловую скорость.

1. Постановка задачи

Пусть на каждом кадре алгоритм обработки изображения выдаёт кватернион q_{meas} . Этот кватернион является шумным измерением ориентации аппарата. Требуется получить уточнённую ориентацию q и оценку угловой скорости ω , чтобы можно было прогнозировать состояние между кадрами.

Важное ограничение: без гироскопа угловая скорость не измеряется напрямую. Она восстанавливается по изменению последовательных кватернионов. Поэтому ориентация обычно оценивается лучше, чем ω .

МЕKF — multiplicative extended Kalman filter. В нём полный кватернион хранится как номинальное состояние, а фильтр Калмана работает с малой трёхмерной ошибкой ориентации. Это лучше, чем напрямую фильтровать четыре компоненты кватерниона, поскольку кватернион должен оставаться единичным.

2. Соглашения о кватернионах

В коде используется гамильтонов кватернион в порядке $[w, x, y, z]$. Произведение обозначается $\otimes$. Кватернионы q и $-q$ описывают одну физическую ориентацию.

```
q = [w, x, y, z]
q_true =  $\delta q \otimes q_{\text{nominal}}$ 
q_error = q_measurement  $\otimes$  inverse(q_predicted)
```

При переносе кода на реальные данные нужно проверить порядок компонентов $[w, x, y, z]$ или $[x, y, z, w]$, направление преобразования $\text{body} \rightarrow \text{reference}$ или $\text{reference} \rightarrow \text{body}$, а также порядок умножения. Несогласованность этих соглашений даёт систематически неправильное вращение.

3. Кватернион как представление вращения

Вращение на угол α вокруг единичной оси u задаётся кватернионом:

$$q(\alpha, u) = [\cos(\alpha/2), u_x \sin(\alpha/2), u_y \sin(\alpha/2), u_z \sin(\alpha/2)].$$

Если задан rotation vector $\phi = \alpha u$, то функция `rotvec_to_quaternion` строит этот кватернион. Для малых углов используется приближение:

$$\delta q(\phi) \approx [1, \phi_x/2, \phi_y/2, \phi_z/2].$$

Нормализация после операций необходима из-за численных ошибок: $q \leftarrow q / \|q\|$.

4. Модель углового движения

4.1. Состояние

Номинальное состояние фильтра состоит из кватерниона и угловой скорости:

$$X = (q, \omega).$$

Но error-state для ковариации имеет шесть компонент:

$$\delta x = [\delta\theta_x, \delta\theta_y, \delta\theta_z, \delta\omega_x, \delta\omega_y, \delta\omega_z]^T.$$

$\delta\theta$ — малая ошибка ориентации в радианах, $\delta\omega$ — ошибка оценки угловой скорости в rad/s. Кватернион не складывается с поправкой аддитивно; поправка вводится умножением на

малый кватернион.

4.2. Модель угловой скорости без гироскопа

Так как гироскопа нет, ω моделируется как медленно меняющаяся величина:

$$\omega_{k+1} = \omega_k + w\omega,$$

где $w\omega$ — случайное изменение угловой скорости. Это random walk-модель. Она не утверждает, что скорость абсолютно постоянна; она задаёт допустимую степень её изменения.

В непрерывном времени: $\dot{\omega} = w\omega$. Параметр `sigma_omega` задаёт интенсивность этого шума. Большое значение позволяет быстро реагировать на манёвр, но делает оценку более шумной. Малое значение даёт плавную скорость, но может вызвать запаздывание.

4.3. Распространение ориентации

За интервал dt угловая скорость создаёт rotation vector:

$$\Delta\theta = \omega dt.$$

Из него строится приращение ориентации $\delta q = q(\Delta\theta)$. Прогноз номинального кватерниона:

$$q_{k+1|k} = \delta q \otimes q_k.$$

Именно это выполняется в `predict()`:

```
delta_angle = self.omega * dt
delta_q = rotvec_to_quaternion(delta_angle)
self.q = q_normalize(q_multiply(delta_q, self.q))
```

4.4. Линеаризованная модель ошибки

Для малых ошибок используем приближение:

$$\delta\theta_{k+1} = \delta\theta_k + dt \delta\omega_k, \delta\omega_{k+1} = \delta\omega_k + \text{noise}.$$

Переходная матрица:

$$F = \begin{bmatrix} I_3 & dt I_3 \\ 0_3 & I_3 \end{bmatrix}.$$

Ковариация прогноза рассчитывается стандартно:

$$P_{k+1|k} = F P_k F^T + Q.$$

4.5. Матрица шума Q

Для модели случайного углового ускорения используется:

$$Q = \sigma_{\omega}^2 \begin{bmatrix} (dt^3/3)I_3 & (dt^2/2)I_3 \\ (dt^2/2)I_3 & dt I_3 \end{bmatrix}.$$

Верхний блок описывает накопление неопределённости ориентации, нижний — неопределённость угловой скорости, а внедиагональные блоки показывают их корреляцию.

5. Измерительная модель по кватерниону

После прогноза приходит `q_meas` от изображения. Сначала исправляется знак:

```
if dot(q_measurement, self.q) < 0:
    q_measurement = -q_measurement
```

Это не меняет физическую ориентацию, но предотвращает искусственный скачок на графике из q в $-q$.

Далее вычисляется относительное вращение между измерением и прогнозом:

$$q_e = q_{\text{meas}} \otimes \text{inverse}(q_{\text{pred}}).$$

Кватернион ошибки переводится в rotation vector $y = \delta\theta_e$. Для малой ошибки его можно считать измерением трёхмерной ошибки ориентации:

$$y \approx H \delta x + v, H = \begin{bmatrix} I_3 & 0_3 \end{bmatrix}.$$

Именно поэтому измерение напрямую наблюдает ориентацию, но не наблюдает ω . Угловая скорость корректируется косвенно через корреляцию P между $\delta\theta$ и $\delta\omega$.

6. Шаг коррекции Калмана

Матрица инновации:

$$S = HPH^T + R.$$

Коэффициент Калмана:

$$K = PH^T S^{-1}.$$

Поправка error-state:

$$\delta x = Ky.$$

Она разделяется на $d\theta$ и $d\omega$. $d\theta$ применяется к кватерниону мультипликативно:

$$q \leftarrow \delta q(d\theta) \otimes q.$$

Угловая скорость корректируется аддитивно:

$$\omega \leftarrow \omega + d\omega.$$

$R = \text{sigma_measurement}^2 I_3$ — ковариация ошибки кватерниона, представленная в радианах. Если алгоритм по изображению неточен, sigma_measurement нужно увеличить.

7. Отбрасывание выбросов через NIS

Для контроля качества измерения используется normalized innovation squared:

$$NIS = y^T S^{-1} y.$$

Если NIS больше порога, измерение не применяется. В коде используется порог 11.34 для трёхмерной инновации:

```
nis = innovation @ solve(S, innovation)
if nis > gate_threshold:
    return False, innovation, nis
```

Так можно отбрасывать плохие кадры, неверное распознавание цели и резкие выбросы PnP или image-based алгоритма.

8. Почему используется Joseph form

Наивная формула $P \leftarrow (I - KH)P$ может из-за ошибок округления нарушить симметрию или положительную полуопределённость ковариации. Поэтому используется Joseph form:

$$P^+ = (I - KH)P(I - KH)^T + KRK^T.$$

После вычислений код дополнительно симметризует матрицу:

```
self.P = 0.5 * (self.P + self.P.T)
```

9. Сброс ошибки MEKF

После применения $d\theta$ номинальный кватернион уже исправлен, поэтому локальная ошибка ориентации сбрасывается к нулю. Однако ковариацию нужно преобразовать в новую локальную систему ошибки. В коде используется приближённая reset-Jacobian:

$$G\theta \approx I_3 - 1/2 [d\theta]_{\times}.$$

После этого: $P \leftarrow G P G^T$. Для малых поправок этого первого порядка достаточно; при больших ошибках начальной ориентации нужно быть особенно внимательным к convention и линеаризации.

10. Разбор основных функций кода

q_multiply

Реализует гамильтоново произведение. Оно объединяет два вращения. Порядок важен: $q1 \otimes q2$ обычно означает сначала применить $q2$, затем $q1$, если используется активная convention.

rotvec_to_quaternion

Преобразует ось-угол в кватернион. Используется и для прогноза по ωdt , и для внесения малой поправки $d\theta$.

quaternion_to_rotvec

Преобразует ошибку кватерниона в трёхмерную инновацию. Перед этим выбирается кратчайшее вращение через условие $q_0 \geq 0$.

predict

Прогнозирует q , затем переносит P вперёд с матрицами F и Q . Здесь нет входа гироскопа: ω берётся из внутренней оценки фильтра.

update

Сравнивает $q_measurement$ с прогнозом, строит инновацию, проверяет NIS, вычисляет K , корректирует q и ω , обновляет и сбрасывает ковариацию.

11. Почему кватернион может быть плавным, а ω — шумной

Ориентация получается интегрированием угловой скорости, а скорость восстанавливается по разности ориентаций:

$$\omega \approx \Delta\theta/dt.$$

Если в двух соседних кватернионах присутствует шум n_k , то скорость содержит $(n_k - n_{k-1})/dt$. Поэтому маленький шум ориентации усиливается при малом dt . Это фундаментальное ограничение gyroless-схемы.

Чтобы сделать ω плавнее, обычно уменьшают σ_{ω} и/или увеличивают $\sigma_{\text{measurement}}$. Цена — большее запаздывание и менее точное отслеживание резких манёвров.

12. Смысл синтетического теста

В тесте создаётся истинная ориентация q_true и истинная скорость ω_true . Скорость медленно изменяется случайным процессом. К истинному кватерниону добавляется случайный rotation-vector noise, имитирующий ошибку определения ориентации по изображению. MEKF получает только это зашумлённое $q_measurement$.

Критерии теста: конечная ошибка ориентации меньше 2 градусов и ошибка угловой скорости меньше 1 градуса в секунду. Эти пороги не являются требованиями миссии; они нужны только для проверки работоспособности примера.

13. Что проверить на реальных данных

- Порядок компонент q : $[w, x, y, z]$ или $[x, y, z, w]$.
- Направление преобразования: $body \rightarrow reference$ или $reference \rightarrow body$.
- Единицы измерения: угловая скорость в rad/s , углы в радианах внутри фильтра.
- Реальный dt по timestamp каждого изображения.
- Нормализацию каждого входного кватерниона.
- Исправление знака q и проверку скачков $q \rightarrow -q$.
- NIS, innovation и качество изображения на каждом кадре.

- Согласованность фильтра с тем, что именно выдаёт image-based методика: абсолютную или относительную ориентацию.

14. Начальные настройки

Параметр	Начальный вариант	Смысл
sigma_omega	0.5–1.0 deg/s	Допустимое изменение угловой скорости
sigma_measurement	реальная ошибка метода	Доверие к кватерниону изображения
P $\theta\theta$	10 deg ²	Начальная неопределённость ориентации
P $\omega\omega$	5–20 (deg/s) ²	Начальная неопределённость скорости
gate_threshold	11.34	Контроль выбросов для 3D инновации

15. Итоговая схема алгоритма

```

q, omega, P = initialize()

for each image:
    dt = timestamp - previous_timestamp

    # Prediction
    q = delta_q(omega * dt) ⊗ q
    P = F P FT + Q

    # Measurement
    q_meas = quaternion_from_image(image)
    q_error = q_meas ⊗ inverse(q)
    innovation = log_map(q_error)

    # Gating
    if NIS <= threshold:
        dx = K innovation
        q = delta_q(dx[0:3]) ⊗ q
        omega = omega + dx[3:6]
        P = covariance_update_and_reset(P)

```

Для сближения аппаратов этот MEKF может выдавать уточнённую относительную ориентацию и оценку относительной угловой скорости. Однако для полного управления сближением отдельно нужна относительная позиция и скорость, полученные из изображения, дальномера или другой навигационной системы.

Литература и справочные материалы

Материалы NASA и публикации по MEKF рассматривают мультипликативное представление ошибки ориентации, кватернионную кинематику и оценку ориентации космических аппаратов. В частности, полезны работы Markley по представлениям ошибки ориентации и материалы NASA по методам оценки ориентации.

Ссылки: NASA NTRS, “Attitude Error Representations for Kalman Filtering”; NASA NTRS, “Nonlinear Attitude Filtering Methods”; NASA NTRS, “Quaternion Normalization in Spacecraft Attitude Determination”.